

HMS Veterinary SaaS

Complete Deployment Guide v2.0

Docker · AWS EC2 · MongoDB Atlas · GitHub Actions

Ubuntu 24.04 · ap-south-1 (Mumbai) · Testing Environment

- React 19
- Node.js
- MongoDB Atlas
- Docker
- AWS ECR
- GitHub Actions

ACTUAL DEPLOYMENT VALUES	
EC2 Public IP	13.232.125.254
EC2 User	ubuntu
EC2 AMI	Ubuntu Server 24.04 LTS
EC2 Region	ap-south-1 (Mumbai)
ECR Registry	232261469028.dkr.ecr.ap-south-1.amazonaws.com
MongoDB Cluster	cluster0.lponosn.mongodb.net
MongoDB DB	hms_db
MongoDB User	mayank_db_user
SSH Key	hms-Key.pem
Frontend URL	http://13.232.125.254
Backend API	http://13.232.125.254:5000

Table of Contents

1.	Project Overview
2.	Architecture Summary
3.	Prerequisites
Step 1	MongoDB Atlas Setup
Step 2	AWS IAM User Setup
Step 3	AWS ECR — Create Image Repositories
Step 4	Launch EC2 Instance
Step 5	Bootstrap EC2 — Docker & AWS CLI (Ubuntu 24.04)
Step 6	Configure Environment Variables on EC2
Step 7	Add GitHub Actions Secrets
Step 8	Project File Reference (all config files)
Step 9	First Deploy
Step 10	Verify the Deployment
—	Ongoing Workflow
—	Troubleshooting
—	Production Checklist

1. Project Overview

WebArclight Veterinary HMS is a multi-tenant SaaS for veterinary clinics. This guide reflects the actual tested deployment on AWS EC2 with real values.

Layer	Technology
Frontend	React 19 + Vite + TailwindCSS
Backend	Node.js + Express REST API
Database	MongoDB Atlas — Cluster0 (M0 Free, ap-south-1)
File Store	AWS S3 + Cloudinary
Payments	Razorpay
Email	Nodemailer — Gmail SMTP (smartvetdr@gmail.com)
Container	Docker + Docker Compose
Registry	AWS ECR (232261469028.dkr.ecr.ap-south-1.amazonaws.com)
Hosting	AWS EC2 Ubuntu 24.04 t3.small — 13.232.125.254
CI/CD	GitHub Actions

2. Architecture Summary

```
git push origin main/develop
|
v
GitHub Actions
|-- build hms-backend --> 232261469028.dkr.ecr.ap-south-1.amazonaws.com/hms-backend
`-- build hms-frontend --> 232261469028.dkr.ecr.ap-south-1.amazonaws.com/hms-frontend
|
v SSH ubuntu@13.232.125.254
AWS EC2 (Ubuntu 24.04, t3.small)
|-- hms_frontend nginx:80 --> http://13.232.125.254
`-- hms_backend node:5000 --> http://13.232.125.254:5000
| |
v v
MongoDB Atlas Cloudinary / S3
cluster0.lponosn dulxydos7
```

3. Prerequisites

- GitHub account with HMS repo pushed
- AWS account — IAM user hms-cicd-user with ECR access
- MongoDB Atlas account — <https://cloud.mongodb.com> (Project: HMS)
- AWS CLI installed locally
- Docker Desktop installed locally
- SSH key: hms-Key.pem at /Volumes/Testing/hms/hms-Key.pem

1 MongoDB Atlas Setup

ACTUAL DEPLOYMENT VALUES

Organization	MAYANK's Org - 2025-08-28
Project	HMS
Cluster	Cluster0 (M0 Free Tier)
Region	AWS / Mumbai (ap-south-1)
DB Users	hms_user (HMS Testing Server), mayank_db_user
Whitelisted IP	13.232.125.254/32 – Active

1.1 Create a cluster (if fresh setup)

- <https://cloud.mongodb.com> -> Build a Database -> M0 Free Tier
- Provider: AWS | Region: ap-south-1 (Mumbai)
- Name: Cluster0 -> Create

1.2 Create database user

- Left sidebar -> Security -> Database & Network Access
- Database Access tab -> + Add New Database User
- Method: Password | Username: hms_user
- Click Autogenerate Secure Password -> Copy and save it
- Built-in Role: Atlas admin -> Add User

i mayank_db_user already exists and is used in backend .env — no need to recreate.

1.3 Whitelist EC2 IP

- Left sidebar -> Security -> Database & Network Access
- Network Access tab -> + Add IP Address
- IP: 13.232.125.254/32 | Comment: HMS_EC2_IPAddress -> Confirm

! Remove 0.0.0.0/0 entry before going to production.

1.4 Get connection string

- Left sidebar -> Database -> Connect on Cluster0
- Drivers -> Node.js 5.5 or later -> copy URI

```
mongodb+srv://mayank_db_user:@cluster0.lponosn.mongodb.net/hms_db?retryWrites=true&w=majority
```

2 AWS IAM User Setup

2.1 Create IAM user for CI/CD

- AWS Console -> IAM -> Users -> Create User
- Username: hms-cicd-user
- Attach policy: AmazonEC2ContainerRegistryPowerUser
- Click Create user

2.2 Generate access keys

- Click hms-cicd-user -> Security credentials -> Create access key
- Use case: Application running outside AWS
- Download CSV — save AWS_ACCESS_KEY_ID and AWS_SECRET_ACCESS_KEY

! You cannot view the secret key again after leaving this page.

2.3 Create EC2 IAM Role

- IAM -> Roles -> Create Role -> AWS service -> EC2
- Attach policy: AmazonEC2ContainerRegistryReadOnly
- Role name: Hms-ec2-ecr-role -> Create role

if you have full IAM access you can skip creating a separate user and use your own access keys, but a dedicated user is safer — limits blast radius if keys leak.

3 AWS ECR — Create Image Repositories

ACTUAL DEPLOYMENT VALUES	
<code>hms-backend</code>	<code>232261469028.dkr.ecr.ap-south-1.amazonaws.com/hms-backend</code>
<code>hms-frontend</code>	<code>232261469028.dkr.ecr.ap-south-1.amazonaws.com/hms-frontend</code>
<code>ECR_REGISTRY</code>	<code>232261469028.dkr.ecr.ap-south-1.amazonaws.com</code>

To create from scratch (if needed):

```
aws configure
# AWS Access Key ID:
# AWS Secret Access Key:
# Default region: ap-south-1
# Default output: json

aws ecr create-repository --repository-name hms-backend --region ap-south-1
aws ecr create-repository --repository-name hms-frontend --region ap-south-1
```

From the JSON output, your `ECR_REGISTRY` is the `repositoryUri` prefix (before `/hms-backend`).

4 Launch EC2 Instance

ACTUAL DEPLOYMENT VALUES	
AMI	Ubuntu Server 24.04 LTS ami-006f82a1d5a27da54
Type	t3.small (2 vCPU, 2 GB RAM)
Public IP	13.232.125.254
Private IP	10.0.1.99
Key pair	hms-Key.pem
IAM profile	Hms-ec2-ecr-role
VPC	vpc-092ae01c525a0ed7a (hms-server)
Subnet	hms_Pub_SN (ap-south-1a)

Security Group inbound rules

Port	Source	Purpose
22	0.0.0.0/0	SSH
80	0.0.0.0/0	Frontend React app (nginx)
5000	0.0.0.0/0	Backend Express API

! Auto-assign Public IP must be ENABLED — required for GitHub Actions SSH and dev team access.

SSH into instance

```
chmod 400 /Volumes/Testing/hms/hms-Key.pem
ssh -i /Volumes/Testing/hms/hms-Key.pem ubuntu@13.232.125.254
```

! Username is 'ubuntu' (NOT ec2-user) — this is Ubuntu 24.04 not Amazon Linux.

5 Bootstrap EC2 — Docker & AWS CLI (Ubuntu 24.04)

! Use apt, NOT yum. This is Ubuntu 24.04 — yum is not available.

```
# Update packages
sudo apt-get update -y

# Install Docker
sudo apt-get install -y docker.io
sudo systemctl start docker
sudo systemctl enable docker
sudo usermod -aG docker ubuntu

# Install Docker Compose plugin
sudo mkdir -p /usr/local/lib/docker/cli-plugins
sudo curl -SL https://github.com/docker/compose/releases/latest/download/docker-compose-linux-x86_64 \
-o /usr/local/lib/docker/cli-plugins/docker-compose
sudo chmod +x /usr/local/lib/docker/cli-plugins/docker-compose

# Install AWS CLI via snap (apt package NOT available on Ubuntu 24.04)
sudo snap install aws-cli --classic

# Create working directory
mkdir -p ~/hms

# Verify
docker --version && docker compose version && aws --version

exit # log out so docker group takes effect
```

SSH back in and verify Docker runs without sudo:

```
ssh -i /Volumes/Testing/hms/hms-Key.pem ubuntu@13.232.125.254
docker ps # empty list, no permission error
```


6

Configure Environment Variables on EC2

Copy .env from local machine to EC2

```
scp -i /Volumes/Testing/hms/hms-Key.pem \  
/Volumes/Testing/hms/HMS_Milestone-2/backend/.env \  
ubuntu@13.232.125.254:~/hms/.env
```

Complete backend .env (production values)

```
# Server  
PORT=5000  
NODE_ENV=production  
  
# MongoDB Atlas – Cluster0 | ap-south-1 | User: mayank_db_user  
MONGO_URI=mongodb+srv://mayank_db_user:ZFwhxCjU5JxL4uR@cluster0.lponosn.mongodb.net/hms_db?retryWrites  
=true&w;=majority  
  
# Auth  
JWT_SECRET=hms_super_secret_jwt_2026_webarclight_xK9mP2qR  
  
# Super Admin OTP  
SUPER_ADMIN_MOBILE=7500447866  
  
# CORS – Testing EC2 IP  
FRONTEND_URL=http://13.232.125.254  
  
# Razorpay (fill when needed)  
RAZORPAY_KEY_ID=  
RAZORPAY_KEY_SECRET=  
  
# AWS S3 (fill when needed)  
AWS_REGION=  
AWS_ACCESS_KEY_ID=  
AWS_SECRET_ACCESS_KEY=  
S3_BUCKET=  
  
# Gmail SMTP  
SMTP_HOST=smtp.gmail.com  
SMTP_PORT=587  
SMTP_EMAIL=smartvetdr@gmail.com  
SMTP_PASSWORD=agpyyombwixmkpsq  
FROM_NAME="HMS Platform"  
FROM_EMAIL=smartvetdr@gmail.com  
  
# Cloudinary  
CLOUDINARY_CLOUD_NAME=dulxydos7  
CLOUDINARY_API_KEY=224328736491543  
CLOUDINARY_API_SECRET=0YoR8Db3YRo7tKZAQxMJ7lI6ao
```

Frontend .env (local file — baked at build time)

Location: /Volumes/Testing/hms/HMS_Milestone-2/frontend/main-app/.env

```
VITE_BACKEND_URL=http://13.232.125.254:5000/api/v1
```

i This value is passed as --build-arg VITE_API_URL in GitHub Actions. It is NOT read from file at runtime — update the VITE_API_URL GitHub Secret if EC2 IP changes.

```
chmod 600 ~/hms/.env
```

7 Add GitHub Actions Secrets

GitHub repo -> Settings -> Secrets and variables -> Actions -> New repository secret

Secret	Value
AWS_ACCESS_KEY_ID	hms-cicd-user access key
AWS_SECRET_ACCESS_KEY	hms-cicd-user secret key
AWS_REGION	ap-south-1
ECR_REGISTRY	232261469028.dkr.ecr.ap-south-1.amazonaws.com
EC2_HOST	13.232.125.254
EC2_USER	ubuntu (NOT ec2-user)
EC2_SSH_KEY	Full contents of hms-Key.pem (incl. BEGIN/END lines)
VITE_API_URL	http://13.232.125.254:5000

i cat /Volumes/Testing/hms/hms-Key.pem — copy everything to EC2_SSH_KEY secret.

8 Project File Reference

```
HMS_Milestone-2/
|-- backend/
| |-- Dockerfile <- Node.js container (non-root, healthcheck)
| |-- .env <- Production env vars ** DO NOT COMMIT **
| `-- .env.example <- Template for new team members
|-- frontend/main-app/
| |-- Dockerfile <- Vite build + nginx (multi-stage, ~25MB)
| |-- nginx.conf <- SPA routing + asset caching
| `-- .env <- VITE_BACKEND_URL (baked at Docker build time)
|-- docker-compose.yml <- Wires backend + frontend, pulls from ECR
|-- .github/workflows/
| `-- deploy-test.yml <- CI/CD: build -> ECR -> EC2
|-- DEPLOYMENT_GUIDE.md <- This document
`-- HMS_Deployment_Guide.pdf <- PDF version
```

backend/Dockerfile

```
FROM node:20-alpine AS deps
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production

FROM node:20-alpine AS runner
WORKDIR /app
RUN addgroup -S appgroup && adduser -S appuser -G appgroup
COPY --from=deps /app/node_modules ./node_modules
COPY . .
RUN chown -R appuser:appgroup /app
USER appuser
EXPOSE 5000
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s --retries=3 \
CMD wget -qO- http://localhost:5000/health || exit 1
CMD ["node", "src/index.js"]
```

frontend/main-app/Dockerfile

```
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
ARG VITE_API_URL=http://localhost:5000
ENV VITE_API_URL=$VITE_API_URL
RUN npm run build

FROM nginx:1.27-alpine AS runner
COPY --from=builder /app/dist /usr/share/nginx/html
```

```
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
HEALTHCHECK --interval=30s --timeout=5s --start-period=5s --retries=3 \
CMD wget -qO- http://localhost:80 || exit 1
CMD ["nginx", "-g", "daemon off;"]
```

frontend/main-app/nginx.conf

```
server {
  listen 80;
  server_name _;
  root /usr/share/nginx/html;
  index index.html;
  gzip on;
  gzip_types text/plain text/css application/json application/javascript
  text/xml application/xml image/svg+xml;
  location / { try_files $uri $uri/ /index.html; }
  location ~* \.(js|css|png|jpg|jpeg|gif|ico|svg|woff2?)$ {
    expires 1y;
    add_header Cache-Control "public, immutable";
  }
  location = /index.html {
    add_header Cache-Control "no-cache, no-store, must-revalidate";
  }
}
```

docker-compose.yml

```
services:
  backend:
    image: ${ECR_REGISTRY:-hms}/hms-backend:${IMAGE_TAG:-latest}
    build:
      context: ./backend
    container_name: hms_backend
    restart: unless-stopped
    ports: ["5000:5000"]
    env_file: [./backend/.env]
    environment:
      NODE_ENV: ${NODE_ENV:-production}
    healthcheck:
      test: ["CMD", "wget", "-qO-", "http://localhost:5000/health"]
      interval: 30s
      timeout: 5s
      retries: 3
      start_period: 15s

  frontend:
    image: ${ECR_REGISTRY:-hms}/hms-frontend:${IMAGE_TAG:-latest}
    build:
      context: ./frontend/main-app
    args:
      VITE_API_URL: ${VITE_API_URL:-http://localhost:5000}
    container_name: hms_frontend
    restart: unless-stopped
    ports: ["80:80"]
    depends_on:
      backend:
```

```
condition: service_healthy
```

[.github/workflows/deploy-test.yml](#)

```
name: Deploy to AWS EC2 (Testing)
on:
  push:
    branches: [main, develop]

jobs:
  build-and-push:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: aws-actions/configure-aws-credentials@v4
    with:
      aws-access-key-id: ${ secrets.AWS_ACCESS_KEY_ID }
      aws-secret-access-key: ${ secrets.AWS_SECRET_ACCESS_KEY }
      aws-region: ${ secrets.AWS_REGION }
      - uses: aws-actions/amazon-ecr-login@v2
      - run: |
        docker build -t ${ secrets.ECR_REGISTRY }/hms-backend:$GITHUB_SHA ./backend
        docker push ${ secrets.ECR_REGISTRY }/hms-backend:$GITHUB_SHA
      - run: |
        docker build --build-arg VITE_API_URL=${ secrets.VITE_API_URL } \
        -t ${ secrets.ECR_REGISTRY }/hms-frontend:$GITHUB_SHA ./frontend/main-app
        docker push ${ secrets.ECR_REGISTRY }/hms-frontend:$GITHUB_SHA

  deploy:
    needs: build-and-push
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: appleboy/scp-action@v0.1.7
    with:
      host: ${ secrets.EC2_HOST }
      username: ${ secrets.EC2_USER } # ubuntu
      key: ${ secrets.EC2_SSH_KEY }
      source: docker-compose.yml
      target: /home/${ secrets.EC2_USER }/hms
      - uses: appleboy/ssh-action@v1
    with:
      host: ${ secrets.EC2_HOST }
      username: ${ secrets.EC2_USER }
      key: ${ secrets.EC2_SSH_KEY }
      script: |
        cd ~/hms
        aws ecr get-login-password --region ${ secrets.AWS_REGION } \
        | docker login --username AWS --password-stdin ${ secrets.ECR_REGISTRY }
        export ECR_REGISTRY=${ secrets.ECR_REGISTRY }
        export IMAGE_TAG=${ github.sha }
```

```
docker compose pull && docker compose up -d --remove-orphans  
docker image prune -f
```


9

First Deploy

Option A — GitHub push (recommended)

```
git add .
git commit -m "chore: add Docker and CI/CD configuration"
git push origin main
```

GitHub repo -> Actions tab. First run ~4-6 minutes. Subsequent deploys ~90 seconds.

Option B — Manual from local machine

```
export ECR_REGISTRY=232261469028.dkr.ecr.ap-south-1.amazonaws.com

aws ecr get-login-password --region ap-south-1 \
| docker login --username AWS --password-stdin $ECR_REGISTRY

docker build -t $ECR_REGISTRY/hms-backend:latest ./backend
docker build --build-arg VITE_API_URL=http://13.232.125.254:5000 \
-t $ECR_REGISTRY/hms-frontend:latest ./frontend/main-app

docker push $ECR_REGISTRY/hms-backend:latest
docker push $ECR_REGISTRY/hms-frontend:latest

ssh -i /Volumes/Testing/hms/hms-Key.pem ubuntu@13.232.125.254
cd ~/hms
ECR_REGISTRY=232261469028.dkr.ecr.ap-south-1.amazonaws.com IMAGE_TAG=latest docker compose up -d
```

10 Verify the Deployment

On EC2

```
ssh -i /Volumes/Testing/hms/hms-Key.pem ubuntu@13.232.125.254

docker ps # both containers healthy?
curl http://localhost:5000/health # {"status":"Platform is online and running."}
docker logs hms_backend --tail 50
docker logs hms_frontend --tail 20
docker logs hms_backend 2>&1 | grep -i "mongo\|connected"
# Expected: MongoDB Connected: cluster0.lponosn.mongodb.net
```

From browser

Check	URL
Frontend app	http://13.232.125.254
Backend health	http://13.232.125.254:5000/health
API base	http://13.232.125.254:5000/api/v1/

Ongoing Workflow

```
git push origin develop -> GitHub Actions -> ~90 sec -> live on http://13.232.125.254
```

Update .env on EC2

```
ssh -i /Volumes/Testing/hms/hms-Key.pem ubuntu@13.232.125.254  
nano ~/hms/.env # edit value -> Ctrl+O to save  
docker compose restart backend
```

Useful commands

```
docker ps # container status  
docker stats # live CPU/RAM  
docker logs hms_backend -f # stream logs  
docker compose down && docker compose up -d # full restart
```

Troubleshooting

Problem: yum: command not found

```
You are on Ubuntu 24.04 – use apt, not yum:  
sudo apt-get install -y docker.io
```

Problem: awscli not available via apt

```
sudo snap install aws-cli --classic
```

Problem: Container not starting

```
docker logs hms_backend  
docker logs hms_frontend
```

Problem: MongoDB connection refused

- Confirm MONGO_URI has database name: .../hms_db?retryWrites=true...
- Atlas -> Network Access -> confirm 13.232.125.254/32 is Active
- Check Cluster0 is not paused (M0 auto-pauses after 60 days idle)

Problem: GitHub Actions SSH fails

- EC2_USER must be 'ubuntu' (not ec2-user)
- EC2_HOST must be 13.232.125.254
- EC2_SSH_KEY must be full hms-Key.pem contents

Problem: Frontend blank / CORS error

- VITE_API_URL secret must be http://13.232.125.254:5000
- FRONTEND_URL in ~/hms/.env must be http://13.232.125.254
- If EC2 IP changes: update both secrets and redeploy

Production Checklist (before going live)

- Remove 0.0.0.0/0 from MongoDB Atlas Network Access
- Change JWT_SECRET to a cryptographically random 64-char string
- Restrict EC2 Security Group port 22 to your IP only
- Add a domain name and SSL certificate (HTTPS / port 443)
- Upgrade MongoDB Atlas from M0 to M10+ for production load
- Upgrade EC2 from t3.small to t3.medium or larger
- Enable automated MongoDB Atlas backups
- Fill in RAZORPAY_KEY_ID and RAZORPAY_KEY_SECRET for payments
- Fill in AWS_REGION, AWS_ACCESS_KEY_ID, S3_BUCKET for file uploads
- Set NODE_ENV=production (already done)
- Remove 0.0.0.0/0 from MongoDB Atlas IP Access List

WebArclight Veterinary HMS — v2.0

Testing Environment · EC2: 13.232.125.254 · Mumbai (ap-south-1)

React 19 · Node.js · MongoDB Atlas · Docker · AWS ECR + EC2 · GitHub Actions